

NAME: DANIYAL SAEED

SAP ID: 53937

SECTION: BS Data Science

COURSE: COAL

ASSIGNMENT NO 2

Interrupt Handling Imagine that while the CPU is in the middle of executing the ADD instruction (from memory address 101), the "Cancel" button on the toaster is pressed. This action generates a hardware interrupt.

1). According to the lecture, what is the primary advantage of using interrupts for I/O operations?

Sol: Main benefit of employing interrupts for I/O operations

Efficiency and responsiveness.

Rather than busy-wait (poll) an I/O device, the CPU is doing worthwhile work and is awakened only when a device is ready to be serviced. This boosts CPU usage and allows the system to respond more promptly to asynchronous inputs.

2). Describe, step-by-step, what the processor does immediately after detecting the interrupt signal. Your description should include what happens to the current program's execution, the role of the Program Counter (PC), and what the processor does to begin processing the interrupt.

Sol: Step-by-step what the processor does when the hardware interrupt arrives

Context: CPU is running an ADD instruction at address 101 when a hardware interrupt is caused by the toaster's "Cancel" button.

Complete the current instruction (typically).

Most CPUs finish the in-flight instruction (ADD at 101) before dealing with the interrupt (to maintain machine state consistency).

Disable or mask additional interrupts (usual).

The processor can invalidate future interrupts (set interrupt-enable flag to zero) to avoid re-entry while saving state — unless there is explicit support in the architecture for nested interrupts.

Save program state on the stack:

- The value of the Program Counter (PC) that references the subsequent instruction (e.g., 102) is pushed onto the stack (or stored in a hardware-defined location).
- The processor stores also significant status information: flags/condition codes and any registers used by the interrupt mechanism or calling convention (often pushed explicitly by hardware or by the first instructions of the ISR).
- The saved state allows the CPU to restart the interrupted program later precisely where it left off.

Acknowledge & identify the interrupt source

The interrupt controller or CPU identifies the device that generated the interrupt (e.g., reads an interrupt request line or checks an interrupt controller). The CPU uses this information to choose the correct Interrupt Service Routine (ISR) address from the interrupt vector table.

Load ISR address into the PC (transfer control):

The processor places the PC at the entry address of the ISR (from the vector table). Execution resumes at the ISR — the interrupt handler for the "Cancel" button.

Execute the ISR:

The ISR runs code to process the cancel event (abort heating, update device status, reset device interrupt, etc.). The ISR can save/restore other registers it utilizes.

When the ISR completes, an interrupt return instruction (e.g., IRET, RTI) pops saved flags and the saved PC off the stack.

Resume main program:

The CPU resumes execution at the saved PC (address 102), as if the interrupt had not interrupted the flow permanently.

3). Assume this "Cancel" interrupt has a higher priority than a "Timer" interrupt. If the Timer interrupt occurs while the CPU is handling the "Cancel" interrupt, how would a system that uses nested interrupt processing handle this situation?

Sol: Nested interrupts: what if a lower-priority Timer interrupt happens during processing the higher-priority Cancel interrupt

Assumptions: Cancel interrupt has higher priority than Timer interrupt. The CPU is processing the Cancel ISR when the Timer interrupt happens.

Lower-priority interrupt cannot interrupt a higher-priority one.

Since the Cancel ISR is being processed and it has higher priority, the Timer interrupt will not interrupt the Cancel ISR.

What normally occurs:

The Timer interrupt signal is delayed by the interrupt controller (or masked out by the CPU) during the execution of the Cancel ISR.

The Cancel ISR finishes and performs its return-from-interrupt (restores state).

Upon returning, the CPU inspects for delayed interrupts. The Timer interrupt (still delayed) is served next (unless there was another higher-priority interrupt that came in the meantime).

If the system can have true nested interrupts and pre-emption by higher priority only:

A lower-priority interrupt (Timer) will never interrupt a higher-priority ISR. However, a higher-priority interrupt might interrupt a lower-priority ISR. That is, nested interrupt processing allows higher-priority interrupts to interrupt lower-priority handlers, but not vice versa.